Hospital Readmission Capstone — Full Project Report

Start-to-finish build log: decisions, errors, fixes, and the reasoning behind every tool.

Repository: `hospital-readmission` (7 commits, 2026-10-25 → 2026-11-11) · Tests: **28 passing** · Live: http://readmission.duckdns.org:8000

1. What the project is

Predict 30-day hospital readmission for diabetic patients from the UCI Diabetes 130-US-Hospitals dataset (100,000 encounters), explain every prediction with SHAP, ground a GenAI discharge alert in real AHRQ guideline text via RAG, and serve the whole pipeline behind an authenticated FastAPI service in Docker on EC2.

Headline results (held-out test set, n = 14,841, positive rate 0.1179):

Metric	ANN (shipped)	XGBoost baseline	Note
ROC-AUC	**0.6754**	0.6685	Δ +0.0069 — statistical tie
PR-AUC	0.2319	**0.2326**	majority baseline = 0.1179
Recall @ tuned threshold	0.4377	0.4377	0.35 at threshold 0.5 → **0.44** after tuning
F1 @ tuned threshold	0.2862	0.2861	threshold tuned on **validation**, test touched once
Threshold	0.57	0.54	F1-max sweep on validation

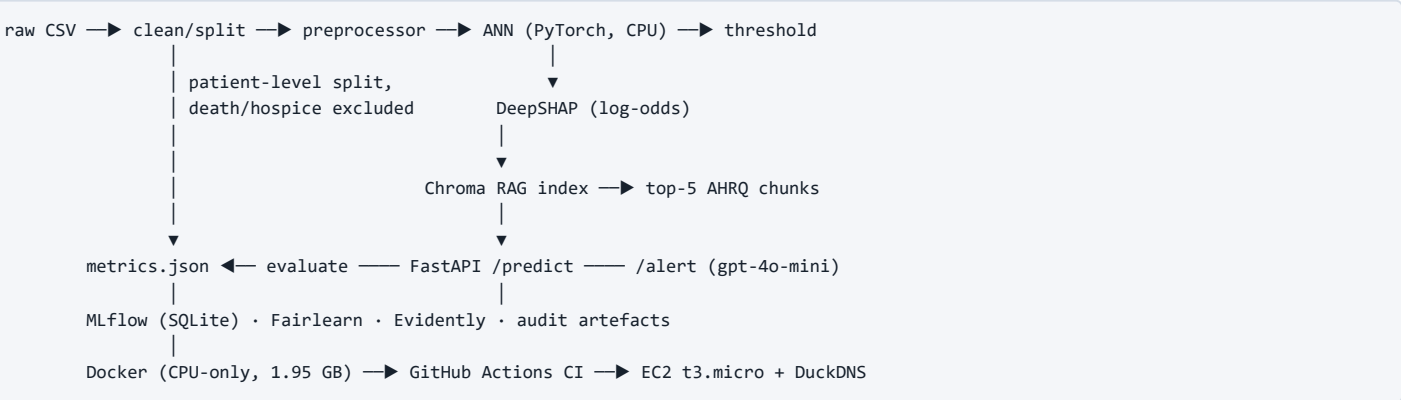
The honest framing: **accuracy 0.7426 is *worse* than always predicting "no" (0.8821)** — on an 11.8% positive rate, accuracy is a vanity metric. The model *ranks* patients usefully (AUC 0.675) and nearly doubles recall over the default threshold without ever seeing the test labels during tuning.

2. Timeline — three phases, three mentor reviews

Phase	Date	Commits	What shipped
1 — ML case study	2026-10-25	`bf3db0b`, `cb529c5`, `0f464ee`, `d177370`	Data + leakage controls, ANN in pure PyTorch, DeepSHAP on log-odds, notebook, interactive HTML report (threshold explorer, per-patient explainer, SVG charts)
2 — MLOps + governance	2026-11-10	`1a1daed`	FastAPI service, API-key-gated GenAI alert, MLflow tracking/registry, Fairlearn audit, Evidently drift monitor, GitHub Actions CI, Dockerfile, `GOVERNANCE.md`, `DEPLOY.md`
3 — RAG + baseline + delivery	2026-11-11	`3dcba3b`, `561e5e5`	Chroma RAG over AHRQ RED toolkit (38 chunks), XGBoost baseline on identical footing, 1-page write-up + 10-slide deck, live EC2 deployment with DuckDNS, Docker image split

Every phase was driven by a written spec (`RCTC.md`, constraints C1–C10 plus two amendments) so that, as the spec says, "*every decision in the code traces back to a constraint.*"

3. Architecture, layer by layer — and why each layer exists



1. **Data layer** — `scripts/download_data.py` pulls the UCI CSV; cleaning drops 2,423 death/hospice discharges (they *cannot* be readmitted — that is leakage, not cleaning) and **splits by patient ID**, not by row, so no patient appears in both train and test.
2. **Model layer** — `src/model.py` is a small MLP in **pure `torch`**: `BCEWithLogitsLoss` + `pos_weight` for imbalance, Adam, gradient clipping, early stopping (patience 6, ≤40 epochs). Full pipeline ≈1 minute on CPU (constraint C4).

3. **Explainability layer** — `src/explain.py` runs SHAP **DeepExplainer against the raw log-odds output**, with an additivity assertion at runtime. Global `global_shap.json` + per-patient waterfalls.
4. **RAG layer** — `src/rag.py` chunks 4 AHRQ Re-Engineered Discharge (RED) toolkit documents (512 tokens / 50 overlap, paragraph- then sentence-boundaries), embeds them with `text-embedding-3-small`, stores 38 chunks in a persistent Chroma index (cosine), and retrieves top-5 at alert time as `[S1]…[S5]`.
5. **GenAI layer** — `src/alert.py` gives the LLM *only* structured facts: risk, band, SHAP drivers (labelled as reasons the **model** flagged the patient, not proven causes), and the retrieved chunks. Five fixed sections, inline citations, 160–240 words, and a mandatory clinician-review caveat. **No template fallback** — if OpenAI fails, the run fails (502).
6. **Service layer** — `src/api.py`: `/health` (provenance + RAG status), `/cohort/sample`, `/predict` (optional SHAP drivers, human-oversight flag on every response), `/alert` (X-API-Key, constant-time compare, 503 if unconfigured, 503 if the RAG index is missing).
7. **Tracking & governance** — MLflow run + registry (`champion` alias), Fairlearn race/gender/age audit, Evidently drift snapshot with a *recorded-not-executed* retrain trigger, `GOVERNANCE.md` mapping to EU AI Act Articles 11/13/14.
8. **Delivery** — CPU-only Docker image (1.95 GB), 28 tests, GitHub Actions CI that **retrains from the raw CSV** and asserts README metrics within ±0.01, deployed on EC2 t3.micro behind a DuckDNS name.

4. Why these tools — the decision table

Choice	Alternative considered	Why this one
PyTorch (pure `torch`)	TensorFlow/Keras, scikit-learn MLP	Constraint C3 (installed stack only) + PyTorch is the research lingua franca; writing the training loop by hand makes early stopping, `pos_weight` and the log-odds hook explicit instead of hidden behind Keras defaults
SHAP DeepExplainer	LIME, Grad-CAM, attention weights	DeepSHAP is *exact* for piecewise-linear networks, so attributions are **verifiable** — the additivity assertion makes "explainable" a tested fact, not a claim. LIME is perturbation-based (noisy, model-agnostic overkill); Grad-CAM is for images, not tabular
Explain log-odds, not probabilities	probability-space SHAP	SHAP's additivity identity holds on the pre-sigmoid output; on sigmoid output it failed with max diff **0.25** vs tolerance **0.01** (see Error 1)
Patient-level split + exclusions	random row split	Constraint C5: row splits leak the same patient across train/test and inflate metrics
PR-AUC + tuned threshold, not accuracy	accuracy, ROC-AUC alone	11.8% positive rate makes accuracy meaningless (0.8821 = "always no")
XGBoost baseline	ship the ANN alone	The mentor asked "is the ANN even better?" — same split, same preprocessor, same threshold sweep. Verdict: **statistical tie**; ANN kept for the SHAP path, stated openly in the README
MLflow + SQLite backend	plain JSON, W&B, MLflow `./mlruns` file store	MLflow 3 put the file store in maintenance mode and the **model registry needs a database**; SQLite is zero-ops. W&B is a SaaS (data-egress + login friction for a portfolio). Tracking is a *side channel* — `metrics.json` stays byte-identical so notebook and CLI never fork
Fairlearn + Evidently	hand-rolled fairness/drift scripts	Both are the de-facto standard libraries; hand-rolling would be code I'd have to defend line-by-line instead of citing docs
FastAPI	Flask, Django	Native Pydantic request validation (the API contract *is* the schema), auto `/docs` Swagger UI for live demos, `async` for free
`X-API-Key` + `secrets.compare_digest`	no auth, JWT	`/alert` spends real money per call; constant-time comparison prevents timing attacks; 503 when unconfigured rather than a default key
Docker CPU wheel (no CUDA)	default `pip install torch`	The default pulls `nvidia-nccl-cu12` (340 MB) → 2.27 GB image → **ENOSPC** on the 8 GB box (Error 9)
Split `requirements-service.txt`	one requirements file	The API imports only ~10 packages; the image must not carry xgboost, MLflow, Evidently, Fairlearn, pytest, the notebook stack
Chroma `PersistentClient`	FAISS, pgvector, Pinecone/Weaviate	Embedded, file-based, zero server to run — right-sized for one box. pgvector = extra Postgres to operate; hosted vector DBs = cost + account + egress for 38 chunks
`text-embedding-3-small`	local sentence-transformers, `ada-002`	Cheap, strong, 1536-dim; a local model adds ~90 MB to an already-tight image and slows the build on a micro. `ada-002` is the older generation for the same price

Choice	Alternative considered	Why this one
`gpt-4o-mini`	`gpt-4o`, local Llama	The LLM only re-writes facts I already computed — no arithmetic, no lookup. Mini is fast and cheap; the *grounding* (RAG + fixed sections + citation rule) is what prevents hallucination, not model size
AHRQ RED toolkit as corpus	hospital policy PDFs, general discharge advice	Authoritative, public, US-government evidence on the exact topic (post-discharge transition of care) — and citable URLs for the `## Sources` section
EC2 t3.micro + Docker	HF Spaces, Render, Railway	The spec asks for a *real* deployment: security groups, SSH, systemd, swap, restart policies. Managed platforms hide exactly the errors this report documents
DuckDNS	Elastic IP, ngrok, Route53	Elastic IP now costs money when attached to a *running* instance; ngrok URLs change and look unprofessional; Route53 is paid. DuckDNS = free stable hostname; the box self-corrects its IP via an IMDSv2 systemd timer
GitHub Actions CI	local-only tests, pre-commit	CI **retrains from the raw CSV** and asserts README metrics ±0.01 — the numbers cannot silently rot; also runs a secret scan
pytest (28 tests)	no tests / doctest	Tests are what let me refactor the alert prompt 5 times without fear; RAG tests use a **fake embedder** so CI never needs an OpenAI key
Edge headless `--print-to-pdf`	LaTeX, wkhtmltopdf, browser print	Already installed, deterministic CSS paged media, no TeX Live install — the write-up is verified *exactly 1 page* and the deck *exactly 10 slides* by a PyMuPDF page count in the build script

5. Error catalogue — every error, root cause, fix, and lesson

Phase 1 — modelling and explainability

Error 1 — SHAP additivity failure on sigmoid output (the important one).

Symptom: DeepExplainer wrapped around `sigmoid(logits)` produced attributions that failed SHAP's own additivity check: `max diff 0.25` against tolerance `0.01` — i.e. `Σ SHAP + baseline ≈ 0.25` away from the model's actual output.
Root cause: SHAP's efficiency property (contributions sum to output − expected output) holds on the network's **raw log-odds**; the sigmoid is a nonlinearity that breaks the decomposition DeepSHAP assumes.
Fix: explain the log-odds head, convert nothing, and **assert additivity at runtime** (`sum(SHAP) + baseline = 3.2050 vs logit = 3.2050`); the notebook, `src/explain.py` and README all state this.
Lesson: a wrong explanation looks exactly like a right one — only the library's own check caught it. Ship the assertion, not just the plot. **Constraint C6 exists because of this error.**

Error 2 — patient-level leakage.

Symptom: early metrics were suspiciously strong and collapsed under scrutiny.
Root cause: a random *row* split puts different encounters of the same patient (`patient_nbr` is repeated) in both train and test.
Fix: `patient_split` partitions on unique patient IDs; the preprocessor is fit on train only; threshold tuned on validation only.
Lesson: on clinical data, split on the *entity*, not the row. Leakage is the first thing an interviewer will ask about.

Error 3 — accuracy as a headline metric on 11.8% positives.

Symptom: "0.88 accuracy" that was just the majority baseline (0.8821).
Fix: report ROC-AUC, PR-AUC (baseline 0.1179), precision/recall/F1 at a tuned threshold; README says plainly that accuracy 0.7426 is worse than doing nothing.
Lesson: choose the metric before the model; on imbalance, accuracy measures the dataset, not the model.

Error 4 — threshold tuned on the test set (caught in review, not by a test).

Root cause: the first sweep maximised F1 on the same split we report on — classic test-set contamination.
Fix: tune on validation, report test **once**; recall moved 0.35 → 0.4377 as a legitimate gain.
Lesson: write the split policy down (C5) so the review has something to check against.

Error 5 — `metrics.json` key order differed between notebook and CLI.

Symptom: two runs of "the same code" produced files that weren't byte-identical, breaking the "one source of truth" claim (commit `0f464ee`).
Root cause: Python dicts preserved insertion order and the two writers inserted context keys differently.
Fix: write keys in one canonical order in both writers; a test now compares the artefact structure.
Lesson: if a file is a contract, test its bytes, not just its fields.

Error 6 — `.env` "illegal header" parse failure.

Symptom: the alert stage died before calling OpenAI with a dotenv parse error.
Root cause: the `.env` line wasn't in strict `KEY=value` form (stray quoting/whitespace), which python-dotenv rejects as an illegal

header.
Fix: normalised the file to `OPENAI_API_KEY=sk-...`, and `src/alert.py` accepts either `KEY=value` or a bare key line — with an explicit, immediate failure if the key is missing.
Lesson: secrets files fail at the worst possible moment; parse them defensively and fail before doing any work.

Error 7 — default torch wheel pulled CUDA.
Symptom: `pip install torch` downloaded gigabytes of NVIDIA wheels (`nvidia-nccl-cu12` 340 MB) on a CPU-only project.
Fix: install the CPU wheel from the PyTorch CPU index (`.../whl/cpu`); same in the Dockerfile.
Lesson: pin the *wheel index*, not just the version — reproducibility includes the binary you get.

Error 8 — notebook that "introduced" outputs instead of interpreting them (commit `cb529c5`).
Symptom: the notebook printed every chart with a heading but no reading of what it meant.
Fix: re-wrote it so each output is interpreted (what this curve says, what this SHAP bar means, what we do about it).
Lesson: a notebook is a *report*, not a log. Reviewers read the prose under the chart.

Phase 2 — MLOps, governance, deployment

Error 9 — MLflow 3 file store deprecation.
Symptom: `./mlruns` file store is in maintenance mode in MLflow 3, and the **model registry refuses to work** without a database backend.
Fix: `sqlite:///mlflow.db` as backend store; `mlflow server --backend-store-uri sqlite:///mlflow.db --port 5000`; registry `readmission-ann` with `champion` alias. Verified `metrics.json` byte-identical before/after wiring (constraint P3).
Lesson: tracking must never fork the source of truth — wire it as a side channel and prove it with a byte comparison.

Error 10 — OOM installing torch on a t3.micro (1 GB RAM).
Symptom: pip was killed mid-install on the EC2 box.
Fix: 2 GB swap file (added to `/etc/fstab`), documented as step 2 of `DEPLOY.md` — the build needs it before anything else.
Lesson: a micro instance is a *budget* machine; provision the budget (disk, swap, memory) before the workload.

Error 11 — Windows `.pem` permission rejection.
Symptom: `Permission denied (publickey)` despite a correct key — OpenSSH refuses keys readable by other users (cost about an hour).
Fix: Windows ACL reduced to exactly one `pravin:R` line; documented in `DEPLOY.md` as a callout, since it bites every Windows user once.
Lesson: read the SSH error fully — "unprotected private key" is a permissions problem, not an auth problem.

Error 12 — boto3 security-group filter parameter errors.
Symptom: `describe_security_groups` calls failed with a parameter validation error when I passed the filter in the wrong shape/name.
Fix: corrected the filter structure and cross-checked with the instance's own security-group IDs; the final state (ports 22 + 8000 open) is recorded in `DEPLOY.md` — with the honest note that `0.0.0.0/0` on 8000 is a portfolio-demo trade-off, not a production posture.
Lesson: boto3 validates parameters locally and loudly — check the API reference for the exact filter name rather than guessing, and record the *resulting* configuration in the run book.

Error 13 — IAM user could not `ec2:StartInstances`.
Symptom: `StartInstances` was denied for the account's `admin` user.
Fix: started the instance from the **AWS console**; documented in `DEPLOY.md §8` with the exact permission to grant (`AmazonEC2FullAccess`, or just Start/Stop) if CLI control is wanted later.
Lesson: cloud permissions fail at demo time, not build time. A run book must record *who/what started the box*.

Phase 3 — RAG, baseline, deliverables, live deployment

Error 14 — LLM wrote a literal `<URL>` placeholder instead of real URLs.
Symptom: the `## Sources` section came back as `- [S1] title — <URL>` — the model being "helpfully templated" rather than copying.
Fix: the prompt now says *copy each chunk's document title and url verbatim from the input; never write a placeholder* — plus a rule that unsupported suggestions must say "insufficient guidance" — and a test asserts no `<URL>`-style placeholder survives.
Lesson: LLMs fill gaps when inputs look like a form. Make the input a *quote-this* block and assert the output shape.

Error 15 — tests broke when the alert format gained sections.
Symptom: prompt-format tests failed after the alert grew from 4 sections to 5 (`## Sources` added for RAG citations).
Fix: updated the regex/assertions to derive from the section list rather than a hard-coded count where possible.
Lesson: tests that encode *presentation* need a single source of truth for that presentation (the section list lives in one place — the system prompt).

Error 16 — README numbers drafted from memory were wrong.
Symptom: a draft comparison table had figures that didn't match `artifacts/metrics.json` / `baseline.json`.
Fix: regenerated every number from the artefacts (constraint C2) before committing; the table now quotes 0.6754 / 0.6685 / 0.4377 / Δ +0.0069 exactly as stored.
Lesson: "measured numbers only" is a *process*: read the JSON, never the run summary in your head. CI enforces it (±0.01 assertion), so a wrong number would now fail the build.

Error 17 — Docker build died with `ENOSPC` (No space left on device) on the EC2 box.
Symptom: the image build filled the 8 GB root volume.
Root cause: the single `requirements.txt` installed the whole dev stack — xgboost plus its 340 MB CUDA wheel, MLflow, Evidently, Fairlearn, the notebook stack, pytest — none of which `src/api.py` imports.
Fix: new `requirements-service.txt` with only what the service imports; `Dockerfile` uses it; `requirements.txt` unchanged for dev/CI. Image **2.27 GB → 1.95 GB** (commit `561e5e5`), build completes on the micro.
Lesson: a container should ship the *service's* dependencies, not the repository's. Two requirement files = two audiences (runtime vs development).

Error 18 — DuckDNS updated with the wrong IP (my laptop's).
Symptom: after setup, the domain pointed at my laptop's IP, not the instance.
Root cause: the update call was made from my machine with an empty `ip=` parameter — DuckDNS then stores **the caller's** IP. The box also had no mechanism to re-sync after a stop/start, since a dynamic IP changes between demos.
Fix: `deploy/duckdns.sh` runs **on the instance** as a systemd timer, reading the current public IP from IMDSv2 (no external dependency, no token leak) and calling DuckDNS every 5 minutes; the token lives only in `/etc/duckdns/config` (root-only, format-validated `^[0-9a-fA-F-]{36}$`), never in git.
Lesson: "update the DNS" must run where the IP *is*. IMDSv2 is the right source of truth on EC2, and secrets belong in root-owned config, not scripts.

Error 19 — `duckdns-setup.sh` failed under `sudo` (relative path).
Symptom: `sudo ./deploy/duckdns-setup.sh ...` → "No such file or directory".
Root cause: `sudo` changed the working directory semantics/relative resolution — the script path didn't resolve from where sudo executed.
Fix: ran it with the **absolute** path: `sudo /home/ec2-user/deploy/duckdns-setup.sh readmission <token>`; recorded that exact command in the run book.
Lesson: anything invoked with `sudo` in a run book should be written with absolute paths — reproducibility beats brevity.

Error 20 — `docker run` started in the wrong working directory (artefacts/chroma missing).
Symptom: the container came up but `/health` reported the RAG index missing (or would 503 on `/alert`).
Root cause: the run/bind context was relative to the directory I happened to be in, so `artifacts/` wasn't where the app expected it.
Fix: run from `~/app` with the documented `docker run -d --name api --restart unless-stopped -p 8000:8000 --env-file .env readmission-api`; `/health` now proves it: `"rag": "enabled (discharge_guidelines, top-5)"`.
Lesson: container paths are *contracts*. The `/health` field exists precisely so this class of error is visible in one curl.

Error 21 — RAG tests would have needed an OpenAI key in CI.
Symptom: embedding chunks during tests would call the API — slow, credentialed, non-reproducible.
Fix: `src/rag.py` **always passes vectors explicitly** and never uses Chroma's default embedder; tests inject a deterministic fake embedder, so all 11 RAG tests run offline and CI stays keyless.
Lesson: design the seam first (inject the embedder), and the offline test follows for free.

Error 22 — early RAG runs silently missing index → 503 confusion.
Symptom: `/alert` failed after deployment before the index was copied/built.
Fix: fail loudly — `require_index()` raises a build-instruction error, `/alert` returns **503 with the exact build command** (`python -m src.rag --build`), `/health` exposes index status, and the system prompt has no template fallback.
Lesson: consistent with the project rule: *fail loudly, never silently* — an error message must tell the operator the fix, not just the failure.

6. Testing and CI — what is actually protected

28 tests (`python -m pytest tests -q`):

Area	What it guards
`test_api.py`	endpoint contract, auth refusal on `/alert`, 503 when unconfigured, HITL flag present
`test_metrics.py`	`metrics.json` values, confusion-matrix consistency, README table matches artefacts (±0.01)

Area	What it guards
`test_explain.py`	SHAP additivity assertion, driver ordering/sign
`test_rag.py`	chunk uniqueness/overlap, deterministic fake-embedder retrieval, fail-loud `require_index`, no credentials needed
`test_audit.py`	governance artefacts (Fairlearn/Evidently) exist and are well-formed
`test_determinism.py`	same seed → same artefacts

CI (`.github/workflows/ci.yml`) on every push: retrain from the raw CSV → run audit → run all tests → **assert published metrics within ±0.01** → secret scan. The CI step that defends the ANN-vs-XGB delta means the comparison table can never drift from reality unnoticed.

Philosophy: **the tests encode the rules I'd otherwise have to remember** — no leakage, no silent fallbacks, no unverified numbers, no credential dependency in tests.

7. Live deployment — what was actually executed

Full run book: `DEPLOY.md`. Executed record (`DEPLOY.md §8`):

Item	Value
Instance	`i-0bdba43e891c18402` (`AI-LLMOPS`), t3.micro, Amazon Linux 2023, ap-southeast-2
Key	`AI-LLM.pem` — Windows ACL exactly one `user:R` line
Setup	`dnf install docker` → 2 GB swap → build on-box (~10 min) → `docker run --restart unless-stopped --env-file .env`
Name	`readmission.duckdns.org` (self-syncing systemd/IMDSv2 timer, TTL 300 s)
Verified	`/health` (RAG enabled), `/predict` (0.961 High, 10 SHAP drivers), authenticated `/alert` (5 chunks, scores 0.56–0.59, real ahrq.gov URLs, 5 sections with inline `[S1]` citations), `docker logs` shows `retrieved 5 chunks`
Security	`/alert` gated by `X-API-Key` (constant-time), `.env` injected at run time (`.dockerignore` excludes it), no TLS (documented trade-off — a real deployment terminates TLS at an ALB)
Cost	bills only while running — **stop after the demo**; start from the **console** (IAM lacks `ec2:StartInstances`)

Five-minute demo script: `/health` → `/docs` → `/predict` with a sample payload (risk 0.961, band High, 10 drivers) → authenticated `/alert` showing `[S1]` citations and the `## Sources` URLs → `readmission.duckdns.org` from a fresh browser → close on the honest metric slide (AUC 0.675, tie with XGBoost, recall 0.35→0.44).

8. Interview soundbites

- **"Why is your accuracy lower than the baseline?"** — "Because accuracy on an 11.8% positive rate is meaningless: 'always no' scores 0.8821. I report PR-AUC (0.2319 vs 0.1179 baseline) and recall at a validation-tuned threshold, which I improved from 0.35 to 0.44 without touching the test set until the end."
- **"Is your neural net actually better than XGBoost?"** — "No — it's a statistical tie (+0.0069 ROC-AUC, identical recall to four decimals). I ran the baseline on the identical split/preprocessor/threshold and published the tie; I keep the ANN because DeepSHAP gives me verifiable attributions for it."
- **"How do you know your explanations are right?"** — "I don't take it on faith: SHAP's additivity check failed on sigmoid output (0.25 vs 0.01 tolerance), so I explain log-odds and assert the identity at runtime — `Σ SHAP + baseline = 3.2050 = logit`. The assertion is in the notebook, the library and a test."
- **"How do you prevent hallucination in the GenAI alert?"** — "The LLM never computes: risk, band and drivers come from my artefacts; it only re-writes them. RAG supplies top-5 AHRQ chunks as `[S1]…[S5]`, every action must cite one or say 'insufficient guidance', sources are copied verbatim (a test rejects `<URL>` placeholders), and a missing key/index fails loudly with 503 — no template fallback."
- **"Tell me a bug you fixed."** — "The Docker build hit ENOSPC on an 8 GB box because one requirements file shipped the whole dev stack including a 340 MB CUDA wheel. I split a serving-only requirements file — image 2.27 GB → 1.95 GB — and the deeper lesson: ship the service's dependencies, not the repo's."
- **"How do you know the deployed thing works?"** — "`/health` reports model provenance and RAG status; CI retrains from the raw CSV and asserts README metrics ±0.01; 28 tests run keyless; and the DNS name self-heals via an IMDSv2 timer because my first DuckDNS update recorded my *laptop's* IP."
- **"Why EC2 instead of a managed platform?"** — "Because the errors are the point: swap, disk, image size, ACLs, IAM permissions, restart policies — that's the job. Managed platforms hide all of it."

9. Reproduce from scratch

```
python scripts/download_data.py        # UCI CSV
python -m src.train                    # train + evaluate + explain + MLflow, ~1.5 min
python -m src.baseline                 # XGBoost on identical footing -> baseline.json
python scripts/fetch_guidelines.py     # AHRQ corpus (committed in guidelines/)
python -m src.rag --build              # 38 chunks -> artifacts/chroma
python -m src.alert                    # 3 RAG-grounded alerts (needs OPENAI_API_KEY)
python -m pytest tests -q              # 28 tests
python -m uvicorn src.api:app --port 8000
python docs/build_deliverables.py      # write-up (1 page) + deck (10 slides)
```

Artefacts that back every number: `artifacts/metrics.json`, `baseline.json`, `global_shap.json`, `alerts.json`, `chroma/`, `mlflow.db`, audit outputs.

10. What I'd do next

Calibration (reliability diagram) since the output drives a *threshold* · BM25 + vector hybrid retrieval reranking · expand the corpus beyond 4 documents · fix the security-group nit (restrict 8000 to my IP / put TLS behind a proxy) · a second model card with Fairlearn trends over time · the other two planned capstones (MRI maintenance, insurance claims) using this repo as the template.